Tuplas e Dicionários

Mutabilidade, chave-valor e catálogo de pacotes

Aula 12 • 21/10/2026 • Linguagem de Programação Python (80h) • CETAM
Profa. Priscila Gonçalves

Visão geral da aula

Data

Quarta-feira, 21/10/2026
Aula 12 • 4 tempos
Laboratório de informática

Habilidade técnica

Aplicar os conceitos de estrutura de dados, organização e acesso de dados, com domínio de conhecimento.

Projeto

Catálogo de pacotes da agência em dicionário

Habilidade técnica

- Aplicar os conceitos de estrutura de dados com domínio
- Criar e acessar tuplas e dicionários
- Comparar lista, tupla e dicionário
- Percorrer dicionários com .items()

Objetos de conhecimento

- Estruturas de dados em Python: listas, tuplas e dicionários
- Hoje: tuplas e dicionários
- Mutabilidade e chave-valor
- Métodos e funções: get, keys, values e items

Roteiro dos 4 tempos

1º tempo

Aula expositiva: tuplas e dicionários

2º tempo

Comparação: lista × tupla × dicionário, mutabilidade e chave-valor

3º tempo

Dicionário de destinos e percurso com .items()

4º tempo

Desafio: catálogo de pacotes da agência

1

1º tempo • Tuplas e dicionários

Aula expositiva e dialogada

Retomada da Aula 11

- Listas, índices e fatiamento
- append, remove, sort e len
- Percurso com for
- Listas paralelas e média de gastos
- Hoje: tuplas e dicionários

Por que outras estruturas?

- A lista serve para muitos valores, mas nem sempre basta
- Dados que não devem mudar pedem outra estrutura
- Dados com nome (destino, preço, dias) pedem rótulos
- Tupla: valores fixos; dicionário: valores com nome
- Escolher bem a estrutura deixa o código mais claro

O que é uma tupla?

- Tupla: coleção ordenada e imutável
- Escrita entre parênteses: (a, b, c)
- Acesso por índice e fatiamento, como na lista
- Não permite alterar, adicionar ou remover itens
- Boa para dados fixos: coordenadas, datas e pares

Criando tuplas

```
coord = (-3.1, -60.0)
dias = (1, 2, 3)
vazia = ()
um = (5,)
print(len(dias))   # 3
```

- Parênteses e vírgulas
- Tupla de um item: vírgula final
- len funciona como na lista

Acessando tuplas

```
pacote = ("Parintins", 3, 850.0)
print(pacote[0])    # Parintins
print(pacote[-1])   # 850.0
print(pacote[1:])   # (3, 850.0)
```

- Índices e fatias, como na lista
- -1 é o último item
- A fatia devolve outra tupla

Tuplas são imutáveis

```
t = (1, 2, 3)
t[0] = 10   # TypeError

# Para "mudar", crie outra tupla
t = (10, 2, 3)
```

- Atribuir por índice dá TypeError
- Reatribuir a variável é permitido
- Garante que o dado não muda

Desempacotando uma tupla

```
destino, dias, preco = ("Tefé", 4, 600.0)
print(destino)
print(dias)
print(preco)
```

- Cada variável recebe um item
- Quantidade precisa coincidir
- Muito usado em for e retornos

O que é um dicionário?

- Dicionário: coleção de pares chave-valor
- Escrito entre chaves: {chave: valor}
- Acesso pela chave, não pelo índice
- Chaves únicas; valores de qualquer tipo
- É mutável: adicionar, alterar e remover

Criando um dicionário

```
pacote = {
    "destino": "Parintins",
    "dias": 3,
    "preco": 850.0
}
print(pacote)
```

- Chave e valor separados por :
- Pares separados por vírgula
- Chaves únicas

Acessando valores

```
p = {"destino": "Tefé", "preco": 600.0}
print(p["destino"])         # Tefé
print(p.get("preco"))       # 600.0
print(p.get("hotel"))       # None
print(p.get("hotel", "-"))  # -
```

- p[chave] dá KeyError se não existe
- get devolve None ou um padrão
- get é mais seguro

Adicionar, alterar e remover

```
p = {"destino": "Tefé"}
p["preco"] = 600.0   # adiciona
p["preco"] = 650.0   # altera
del p["preco"]       # remove
print(p)
```

- Atribuir a chave nova adiciona
- Atribuir a chave existente altera
- del remove a chave

Erros comuns

- KeyError: acessar uma chave que não existe
- TypeError: tentar alterar um item de tupla
- Tupla de um item precisa de vírgula: (5,)
- Confundir { } (dicionário) com [ ] (lista)
- Esquecer os dois-pontos entre chave e valor

2

2º tempo • Comparação

Lista × tupla × dicionário

Lista × tupla × dicionário

Lista

Colchetes [ ]
Ordenada e mutável
Acesso por índice

Tupla

Parênteses ( )
Ordenada e imutável
Acesso por índice

Dicionário

Chaves { }
Pares chave-valor
Acesso pela chave

Mutabilidade

Mutável

Pode ser alterada depois de criada.
Exemplos: lista e dicionário.

Imutável

Não pode ser alterada.
Exemplos: tupla, str, int e float.

Mutável x imutável na prática

```
l = [1, 2, 3]
l[0] = 10     # funciona
t = (1, 2, 3)
t[0] = 10     # TypeError
```

- Lista aceita a troca
- Tupla recusa a troca
- Escolha conforme a necessidade

Chave-valor

- O dado é encontrado pelo rótulo
- Exemplo do dia a dia: agenda de contatos
- Exemplo da agência: preço de cada destino
- Chave identifica; valor guarda a informação
- Evita decorar posições, como na lista

Mesmo dado, duas estruturas

```
# Lista: lembrar a posição
pacote = ["Tefé", 4, 600.0]
print(pacote[2])

# Dicionário: acesso pelo nome
p = {"destino": "Tefé", "preco": 600.0}
print(p["preco"])
```

- Lista: posição
- Dicionário: nome da chave
- Dicionário é mais legível

Quando usar cada estrutura

- Lista: sequência de itens que muda (gastos)
- Tupla: grupo fixo de valores (destino e dias)
- Dicionário: dados com nome (pacote com campos)
- Pergunte: preciso alterar? preciso de rótulos?
- Combinações são comuns: lista de dicionários

Convertendo entre estruturas

```
t = tuple([1, 2, 3])
l = list((4, 5, 6))
print(t)   # (1, 2, 3)
print(l)   # [4, 5, 6]
```

- tuple() converte para tupla
- list() converte para lista
- Útil para alterar uma tupla

Percorrendo cada uma com for

```
for x in [10, 20]:          # lista
    print(x)
for x in (10, 20):          # tupla
    print(x)
for k in {"a": 1, "b": 2}:  # dicionário
    print(k)
```

- for percorre as três
- No dicionário, percorre as chaves
- Para pares: .items()

Métodos do dicionário

```
p = {"destino": "Tefé", "preco": 600.0}
print(p.keys())
print(p.values())
print(p.items())
```

- keys: as chaves
- values: os valores
- items: pares (chave, valor)

3

3º tempo • Atividade prática

Dicionário de destinos com nome, preço e duração

Atividade prática

- Dicionário de destinos da agência
- Cada destino tem nome, preço e duração
- Percorrer o dicionário com .items()
- Mostrar o catálogo formatado
- Entrega: arquivo .py com comentários

Estrutura do dicionário de destinos

```
destinos = {
    "Parintins": {"preco": 850.0, "dias": 3},
    "Tefé": {"preco": 600.0, "dias": 4},
    "Novo Airão": {"preco": 420.0, "dias": 2}
}
```

- Chave: nome do destino
- Valor: outro dicionário
- Preço e duração em cada um

Acessando os dados

```
print(destinos["Tefé"])
print(destinos["Tefé"]["preco"])
print(destinos["Tefé"]["dias"])
print(len(destinos))   # 3
```

- Dois colchetes: destino e campo
- Tefé: preço 600.0 e 4 dias
- len conta os destinos

Percorrendo com .items()

```
for nome, dados in destinos.items():
    print(nome, "-", dados["dias"], "dias")
```

- .items() devolve pares
- nome recebe a chave
- dados recebe o dicionário interno

Catálogo e soma dos preços

```
for nome, d in destinos.items():
    print(nome, "R$", d["preco"])
total = 0
for d in destinos.values():
    total += d["preco"]
print("Soma: R$", total)
```

- .values() devolve só os valores
- Soma dos preços: 1870.0
- Compare com sum() em listas

Critérios da atividade

- Dicionário com nome, preço e duração de cada destino
- Percurso correto com .items()
- Acesso por chave sem KeyError
- Saída organizada e conferida
- Comentários e nomes de variáveis claros

4

4º tempo • Desafio do projeto

Catálogo de pacotes da agência

Desafio: catálogo de pacotes

- Catálogo de pacotes da agência em dicionário
- Cada pacote tem nome, destino e preço
- Mostrar todos os pacotes do catálogo
- Consultar um pacote pelo nome
- Opcional: cadastrar novo pacote e mostrar o mais barato

Exemplo de solução do desafio

```
catalogo = {
    "Econômico": ("Novo Airão", 900.0),
    "Conforto": ("Tefé", 2100.0),
    "Premium": ("Parintins", 4200.0)
}
nome = input("Pacote: ")
if nome in catalogo:
    destino, preco = catalogo[nome]
    print(destino, "- R$", preco)
else:
    print("Pacote não existe")
```

- Chave: nome do pacote
- Valor: tupla (destino, preço)
- in testa se a chave existe

Dicas para o desafio

- Use o nome do pacote como chave
- Guarde destino e preço em uma tupla ou dicionário
- Use in para evitar KeyError
- Percorra o catálogo com .items()
- Teste com um pacote que existe e outro que não

Fechamento

- Hoje: tuplas, dicionários e chave-valor
- Prática: dicionário de destinos com .items()
- Desafio: catálogo de pacotes da agência
- Dúvidas?
- Obrigada!